

The Herzog–Schönheim conjecture for coset partitions with at most seventeen cells

An unreviewed computer-assisted candidate

Rio Itabe

Independent researcher

Version 0.4.2-review-candidate, 17 August 2026

This manuscript has not been peer reviewed. It presents a computer-assisted proof candidate for a bounded partial result only. The unrestricted Herzog–Schönheim conjecture remains open, and no claim of priority is made.

Abstract

The Herzog–Schönheim conjecture asserts that a nontrivial finite partition of a group into left cosets of finite-index subgroups has two equal subgroup indices. We present an unreviewed computer-assisted proof candidate that the conjecture holds for every partition with at most seventeen cosets. Equivalently, every counterexample requires at least eighteen cells. After index-two descent and four published Margolis–Schnabel obstructions, exact arithmetic enumeration leaves five profiles at length seventeen, all beginning with index four. An index-four cell forces a finite assignment to three non-anchor left-coset boxes. If a cell of ambient index n meets d boxes and has induced index e inside the anchor subgroup, then $nd = 4e$; every box has reciprocal capacity one; and induced indices sharing a box have gcd greater than one. A second exact search rejects all five profiles. The Lean 4 endpoint has no external theorem parameter: the finite-quotient reduction, the two partition identities, the four source-shaped Margolis–Schnabel obstruction arguments, both group-to-search bridges, and both finite searches are proved in its local dependency chain. The finite computations are split into checked-in `decide +kernel` certificates, and the final axiom report contains only `propext`, `Classical.choice`, and `Quot.sound`. Two separately implemented exact-arithmetic Python programs reproduce the supplementary arithmetic and fiber boundaries; they share the same mathematical obstruction specification but import no code from one another. At length eighteen, 39 necessary-condition fiber profiles survive; they are not coset partitions or counterexamples. To our knowledge, the previously published unrestricted cell-count result covered at most seven cells.

1 Introduction and statement

Let G be a group. A finite family of left cosets

$$\mathcal{P} = \{g_i H_i : 1 \leq i \leq r\} \tag{1.1}$$

of finite-index subgroups is a *coset partition* if every element of G belongs to exactly one displayed coset. Herzog and Schönheim posed the conjecture in 1974 [6]: a nontrivial such partition, meaning $r \geq 2$, cannot have all indices $[G : H_i]$ distinct. See Margolis–Schnabel [8] and the maintained problem record [4] for background.

The maintained Erdős Problems record currently adopts this arbitrary-group formulation as Problem 274. The historically asked abelian special case is already known through the

subnormal-subgroup result recorded there. Our claim is a bounded result for the current general-group formulation and is not a new result for the abelian case.

Our candidate partial theorem is the following.

Theorem 1.1. *Let G be an arbitrary group and let (1.1) be a coset partition with $2 \leq r \leq 17$. Then there are distinct i, j such that*

$$[G : H_i] = [G : H_j].$$

Equivalently, every counterexample to the Herzog–Schönheim conjecture requires at least eighteen cells.

Akman and Sissokho proved the corresponding arbitrary-group result for at most seven distinct subgroups [1, Theorem 6]. In a counterexample the indices are distinct, so the same subgroup cannot occur twice. Thus their number of distinct subgroups is exactly the number of cells, and the two formulations are directly comparable.

The enumeration used here imposes the reciprocal-sum and pairwise-gcd conditions together with four non-harmonic patterns from Margolis–Schnabel Propositions 4.2, 4.3, 4.5, and 4.7 [8], and examines the full remaining denominator interval. The five profiles left at seventeen are then subjected to the index-four fiber obstruction developed below.

The theorem is computer assisted, but its trust layers are separated:

- (i) cited articles provide provenance and statement comparison;
- (ii) Lean proves the finite quotient, partition identities, source-shaped obstruction theorems, detector transfers, and group-to-search bridges;
- (iii) the arithmetic and fiber computations are divided into finite `decide +kernel` certificates whose coverage is checked by Lean;
- (iv) deterministic generators reproduce every certificate module byte for byte, but are not premises of the theorem;
- (v) implementation-diverse Python programs separately reproduce the supplementary length-seventeen and length-eighteen diagnostics.

Section 7 states precisely what is and is not formalized.

2 Finite quotient and index-two descent

We first record two reductions that are easy to hide when moving between the literature and the executable search.

2.1 Reduction to a finite group

Lemma 2.1 (Finite quotient). *Let (1.1) be a finite coset partition of an arbitrary group, with every H_i of finite index. There is a finite quotient G/N carrying a coset partition with the same number of cells and the same list of indices.*

Proof. For each i , the normal core

$$\text{core}_G(H_i) = \bigcap_{g \in G} gH_i g^{-1}$$

is the kernel of the action of G on the finite set G/H_i , hence is normal and has finite index. Put

$$N = \bigcap_{i=1}^r \text{core}_G(H_i).$$

Then N is normal and finite index, and $N \leq H_i$ for every i . Each cell $g_i H_i$ is a union of N -cosets, so the cells descend to a partition

$$\{(g_i N)(H_i/N) : 1 \leq i \leq r\}$$

of the finite group G/N . More explicitly, for the quotient map π ,

$$\pi^{-1}((g_i N)(H_i/N)) = g_i H_i.$$

Thus coverage, disjointness, and the number of cells are all preserved. Finally,

$$[G/N : H_i/N] = [G : H_i].$$

□

This is the standard finite reduction in the subject; see also Korec–Znám [7]. It permits finite-group harmonic propositions to be used without changing the profile. The construction, exact cell preimages, partition descent, and complete index-profile preservation are formalized in `FiniteQuotientBridge`.

2.2 Descent past an index-two cell

Lemma 2.2 (Index-two descent). *Suppose $\mathcal{P}$ is a coset partition with $r \geq 2$ and all indices are distinct, and suppose one cell is a coset of an index-two subgroup K . Then $r > 2$, and K has a distinct-index coset partition with $r - 1$ cells.*

Proof. Let the index-two subgroup be K . Translate the entire partition so that its cell is K itself. An index-two subgroup is normal, and the complement of K is one coset aK . Every other cell $g_i H_i$ is disjoint from K , hence is contained in aK .

For $h \in H_i$, both g_i and $g_i h$ lie in aK . It follows that $h = g_i^{-1}(g_i h)$ lies in K , so $H_i \leq K$. Left translation by a^{-1} therefore sends all cells other than K to a partition of K into cosets of the same subgroups H_i . Index multiplicativity gives

$$[G : H_i] = [G : K][K : H_i] = 2[K : H_i]. \quad (2.1)$$

Thus dividing all remaining indices by two preserves their distinctness.

If $r = 2$, the translated family has one cell, so its only subgroup equals K and has index one in K . Equation (2.1) makes the other original index equal to two, the same as the removed cell, contrary to distinctness. Hence $r > 2$, and the translated family is a nontrivial distinct-index partition of K with $r - 1$ cells. □

Now suppose a counterexample to Theorem 1.1 exists, and choose one with the fewest cells among all counterexamples having at most seventeen cells. Lemma 2.2 shows that none of its indices is two. No minimality in the group order is needed.

3 The arithmetic index profile

Write the distinct indices of the chosen minimal-cell counterexample in increasing order:

$$3 \leq a_1 < a_2 < \cdots < a_r. \quad (3.1)$$

The lower bound follows from the index-two descent and the fact that no cell of a nontrivial partition can have index one.

Two standard necessary conditions are recorded in Margolis–Schnabel Lemma 2.3(b,c) [8]:

$$\sum_{i=1}^r \frac{1}{a_i} = 1, \quad (3.2)$$

$$\gcd(a_i, a_j) > 1 \quad (i \neq j). \quad (3.3)$$

For a finite quotient, (3.2) is also the identity obtained by dividing the cardinalities of the partition cells by $|G|$. Condition (3.3) is the familiar fact that cosets of subgroups with coprime finite indices cannot be disjoint.

We call a list satisfying (3.1)–(3.3), together with the four exclusions in Section 4, an *arithmetic candidate*. Every bounded counterexample produces one, but the converse is neither used nor expected.

4 Published harmonic obstructions

A tuple $(d_1, \dots, d_s)$ is G -harmonic if there are subgroups of G with those indices and representatives whose chosen cosets are pairwise disjoint. Every selected subfamily of a coset partition is therefore G -harmonic.

The following four forbidden shapes are consequences of Margolis–Schnabel Propositions 4.2, 4.3, 4.5, and 4.7 [8]. In each row, all displayed residual variables are pairwise coprime.

Source	Forbidden index subtuple	Extra condition
Prop. 4.2	$(2r_1, 2r_2, 2r_3)$	none
Prop. 4.3	$(3r_1, 3r_2, 3r_3, 3r_4)$	none
Prop. 4.5	$(2r_1, 4r_2, 4r_3, 4r_4)$	r_1 odd
Prop. 4.7	$(3, 3r_2, 6r_3, 6r_4, 6r_5)$	r_2 odd

The executable detectors use congruence classes and divide by the common factor. For example, the Proposition 4.5 detector chooses one index congruent to 2 modulo 4, divides it by 2, chooses three indices divisible by 4, divides them by 4, and tests pairwise coprimality of the four residuals. The Proposition 4.7 detector requires the distinguished index 3, a second index congruent to 3 modulo 6 but unequal to 3, and three indices divisible by 6.

Lemma 4.1 (Detector-to-subfamily transfer). *If the sorted index profile of a distinct-index coset partition triggers one of the four detectors, then the partition contains a harmonic subfamily with the corresponding forbidden index tuple.*

Proof. Every selected index value comes from a unique cell because the index map is injective. Selecting the corresponding cells preserves pairwise disjointness. For Propositions 4.2 and 4.3, the common-factor division gives the residuals in the source statements. For Proposition 4.5, the index congruent to 2 modulo 4 cannot equal any of the three indices divisible by 4, and its quotient by 2 is odd. For Proposition 4.7, the distinguished index 3, the nontrivial index congruent to 3 modulo 6, and the three multiples of 6 are all distinct; dividing the

second index by 3 gives an odd residual. Strict increase handles distinctness within each pool. The source notion is unchanged by reordering the selected cells. The detector’s pairwise coprimality test is exactly the remaining source hypothesis. $\square$

It follows that the profile of a counterexample triggers none of the four detectors. Lemma 2.1 supplies the finite setting if the original ambient group was infinite.

The local formal proofs use the source-shaped statements but do not copy the published proofs line by line. Two presentation points are made explicit in the Lean modules. First, the final inference printed in Corollary 3.11 is replaced by a direct parity and finite-cardinality argument in the Proposition 4.3 proof. Second, the penultimate paragraph of the printed Lemma 4.6 repeats the vector $(1, 2, 2, 3, 1, 1)$; consistency with the listed remaining cases requires that paragraph-start occurrence to be $(1, 2, 2, 3, 3, 1)$, distinct from the final survivor $(1, 2, 2, 3, 1, 1)$. The Proposition 4.7 module records and checks this case split explicitly.

5 Complete exact enumeration

We enumerate strictly increasing denominator lists with exact rational arithmetic. At a recursive node, suppose the positive residual reciprocal sum is q , the previous denominator is p , and k denominators remain. If d is the next denominator, then

$$\max\{p + 1, \lceil 1/q \rceil\} \leq d \leq \lfloor k/q \rfloor. \quad (5.1)$$

At the root we also impose $d \geq 3$.

The lower bound follows from strict increase and $1/d \leq q$. For the upper bound, every remaining denominator is at least d , so their reciprocal sum is at most k/d . Hence $q \leq k/d$.

For every integer d in (5.1), the recursion first performs two monotone prefix tests:

- (1) $\gcd(d, a_i) > 1$ for every earlier denominator;
- (2) the extended prefix contains none of the four published obstructions;

It then forms the updated residual $q' = q - 1/d$, which is nonnegative by the lower bound in (5.1), and stores it as a reduced numerator–denominator pair. A zero residual is accepted only when no denominators remain; if terms remain, that child state is rejected immediately. Coprime pairs and forbidden subtuples persist under extension, so both prefix rejection tests are monotone.

Proposition 5.1 (Search completeness). *Every arithmetic candidate of length r is emitted by the exact recursive search at length r .*

Proof. Induct on the number of denominators remaining. The next denominator of a candidate lies in the complete interval (5.1). Conditions (3.3) and the absence of a published obstruction make it pass both monotone guards. Exact subtraction sends the candidate tail to the reduced state representing its reciprocal sum, and strict increase supplies the next previous-denominator invariant. The induction hypothesis then emits the tail. At zero remaining terms, equation (3.2) makes the residual exactly zero, so the completed list is returned. $\square$

This induction is formalized in Lean as `searchComplete`. The proof also establishes the natural-number ceiling and floor bounds used by the implementation and the equality between the rational reciprocal sum and the reduced integer state.

The length-seventeen equality is not a monolithic evaluator call. The complete 1,052-state recursion tree is divided by depth across `ArithmeticSearchCertificateD0` through

`ArithmeticSearchCertificateD17`. A terminal or local admissible- denominator check is discharged by `decide +kernel`; an internal node unfolds one recursion step and rewrites by every imported child theorem. The root theorem is `checked_seventeen_survivors`.

6 The arithmetic boundary and index-four fiber obstruction

6.1 The five arithmetic profiles

The primary arithmetic program uses Python’s exact `fractions.Fraction` arithmetic and recursive traversal. The separately implemented verifier stores a reduced pair of natural numbers and uses an explicit stack; it has its own code for obstruction logic, constants, hashing, and checked ranges. The two programs import no code from one another but share the same mathematical obstruction specification. They agree on every recorded count and endpoint hash.

Obstruction profile	Zero through	First survivor length	Survivors
Propositions 4.2 and 4.3	14	15	28
Propositions 4.2, 4.3, 4.5, 4.7	16	17	5

For the full profile, the primary recursion visits 117, 161, 244, 406, and 1052 nodes at lengths 13, 14, 15, 16, and 17. These implementation-specific counts are retained only as reproducibility checks.

All five length-seventeen survivors share the prefix

$$(4, 6, 8, 12, 16, 18, 24, 30, 32, 36, 40, 42, 48)$$

and have the following four-term tails:

$$\begin{aligned} & (54, 56, 60, 4320), \quad (56, 60, 64, 320), \\ & (56, 60, 80, 160), \quad (56, 60, 96, 120), \\ & (56, 72, 90, 96). \end{aligned} \tag{6.1}$$

The SHA-256 of their canonical sorted JSON encoding is

$$0244dcaa451200177fe26b8ea2b8ab53f3737817004129afbd9969e1bead2d57.$$

In particular, every profile contains the index four. This is a checked fact about (6.1), not a general assertion about coset partitions.

6.2 The three-box assignment

Suppose a distinct-index coset partition has one of the profiles in (6.1). Choose its unique index-four cell and translate the partition so that this cell is the subgroup H itself. No normality of H is assumed. The set of left H -cosets has four elements, and the complement of the anchor therefore consists of three left-coset *boxes*, which we label 1, 2, 3.

Consider another cell gK and write $n = [G : K]$. Let $S(gK)$ be the nonempty set of boxes met by gK , put $d = |S(gK)|$, and set

$$e = [H : H \cap K].$$

The set of left cosets of $H \cap K$ in K injects into the set of left H -cosets, with image exactly the boxes met by gK . Hence it is in bijection with that support, so $d = [K : H \cap K]$. Computing $[G : H \cap K]$ in the two subgroup chains gives

$$nd = 4e. \tag{6.2}$$

If two distinct cells meet the same box, translating that box back to H produces disjoint left cosets of the subgroups $H \cap K$ and $H \cap L$ in H . Cosets of finite-index subgroups with coprime indices always intersect: indeed, the natural map from one subgroup to the coset space of the other is surjective when the two indices are coprime. Consequently

$$\gcd(e_K, e_L) > 1 \quad (6.3)$$

whenever the two rows share a box.

We also need the exact reciprocal capacity of each box. We use the elementary coset-cover inequality: if finitely many left cosets of finite-index subgroups cover a group L , then the sum of their reciprocal indices in L is at least one. To see this, pass to the finite quotient by the intersection of their normal cores and compare cardinalities.

For a fixed box b , the fragments of all cells meeting b cover it. Translating the box to H and applying the cover inequality gives

$$C_b := \sum_{gK: b \in S(gK)} \frac{1}{e_K} \geq 1. \quad (6.4)$$

On the other hand, (6.2) and the ambient reciprocal identity give

$$\sum_{b=1}^3 C_b = \sum_{gK \neq H} \frac{d_K}{e_K} = 4 \sum_{gK \neq H} \frac{1}{[G : K]} = 4 \left(1 - \frac{1}{4}\right) = 3. \quad (6.5)$$

There are three boxes and each capacity is at least one, so

$$C_1 = C_2 = C_3 = 1. \quad (6.6)$$

Equations (6.2), (6.3), and (6.6) define the finite *index-four fiber assignment* forced by the partition: every non-anchor index n chooses a nonempty support $S \subseteq \{1, 2, 3\}$, with $e = n|S|/4$ integral; every box has reciprocal e -sum one; and all induced indices sharing a box have gcd greater than one.

6.3 Kernel depth-first search and split certificates

We now specify the finite model independently of its implementation. Write a profile as

$$x = (4, n_1, \dots, n_s), \quad B = \{0, 1, 2\}.$$

For one row n , the complete choice set is

$$\mathcal{C}(n) = \{(S, e) : \emptyset \neq S \subseteq B, n|S| = 4e\}. \quad (6.7)$$

Thus the seven nonzero three-bit masks enumerate all possible supports, and a mask is retained precisely when $n|S|$ is divisible by four. Let

$$L = \text{lcm}\{e : (S, e) \in \mathcal{C}(n_i) \text{ for some } i\}. \quad (6.8)$$

All the profiles under consideration have positive rows, so every displayed e is positive and divides L .

After t rows, a labelled search state is

$$Q = ((f_0, M_0), (f_1, M_1), (f_2, M_2)). \quad (6.9)$$

Here f_b is the integer-scaled fill already placed in box b , and M_b is the list, with multiplicity, of induced indices already placed there:

$$f_b = \sum_{\substack{i \leq t \\ b \in S_i}} \frac{L}{e_i}. \quad (6.10)$$

The initial state is $((0, []), (0, []), (0, []))$. For a choice (S, e) in the next row, every $b \in S$ is updated by

$$(f_b, M_b) \mapsto \left(f_b + \frac{L}{e}, e :: M_b \right), \quad (6.11)$$

and the other columns are unchanged. A branch is kept exactly when $e > 0$, $e \mid L$, every new fill is at most L , and

$$\gcd(e, u) > 1 \quad \text{for every } u \in M_b \text{ and every } b \in S. \quad (6.12)$$

No quotient by a permutation of the boxes and no state deduplication is used in the theorem-facing checker. It is a direct depth-first search. After all rows are processed, a branch accepts exactly when $f_0 = f_1 = f_2 = L$.

The executable procedure can therefore be summarized as follows.

```
KERNEL-FIBER-DFS(4, n[1], ..., n[s]):
  choices[n] := all nonempty masks S with n*|S| = 4*e
  L := lcm of every induced e in every choices[n]
  DFS(rows, Q):
    if rows is empty:
      return every column fill equals L
    n := first row
    for (S, e) in choices[n]:
      reject if e = 0 or e does not divide L
      reject if a selected fill would exceed L
      reject if gcd(e, u) = 1 for an old u in that box
      if DFS(remaining rows, updated Q): return true
    return false

  return DFS(all rows, ((0, []), (0, []), (0, [])))
```

Each rejection is safe for a declarative assignment forced by a partition. The divisibility guard follows from (6.8). A true prefix cannot overfill, because all increments are nonnegative and its final fill is exactly L by (6.6). The gcd guard is exactly (6.3). Finally, after all rows, an underfilled state cannot satisfy the exact capacity identity. These are the only rejections used by the theorem-facing Lean transition system.

The search terminates because there are finitely many rows, each row has at most seven choices, and the row iterator is structural recursion on the finite tail. Its completeness is the direction needed in the proof.

Proposition 6.1 (Depth-first-search completeness). *Every index-four fiber assignment for x determines a path from the initial state to an accepting branch of the depth-first search.*

Proof. Fix an assignment and process its rows in profile-tail order. Its support in the next row has a three-bit encoding in $\mathcal{C}(n)$ by (6.2). The induced index divides L . For every box, the fill of the processed prefix is at most the final fill L , and every old induced index in the same box satisfies (6.3). Hence the assignment's next transition passes every guard. Induction on the processed prefix therefore supplies one recursive branch. At the terminal depth, (6.6) makes all three fills equal to L , so the branch accepts. $\square$

This proposition asserts only

$$\text{fiber assignment forced by a partition} \implies \text{accepted DFS branch.} \quad (6.13)$$

The reverse construction of a group partition from an accepted branch is not claimed and is not needed. Therefore an empty executable search rules out all group-theoretic assignments, whereas a surviving state is only a necessary- condition object.

The correspondence with the Lean development is explicit.

Mathematical object or step	Lean identifier
Declarative assignment	<code>IndexFourFiberAssignment</code>
Ordered mask witness	<code>OrderedFiberMaskAssignment4</code>
Group assignment to ordered witness	<code>IndexFourFiberAssignment.toOrderedMaskAssignment4</code>
Complete row choices	<code>kernelFiberChoices4</code>
Column and three-column update	<code>kernelUpdateFiberColumn,</code> <code>kernelUpdateFiberColumns</code>
Depth-first search and common scale	<code>kernelFiberDFS4, kernelFiberScale4</code>
Assignment-to-DFS completeness	<code>kernelFiberDFS4_complete_of_orderedAssignment</code>
Five negative profile endpoints	<code>kernelFiberProfile1_unsat</code> through <code>kernelFiberProfile5_unsat</code>
Combined obstruction	<code>seventeenIndexFourAssignmentObstruction_kernel</code>

The monolithic closed computation is too large for practical kernel reduction, so its finite recursion tree is split. A generated leaf contains at most 6,000 visited DFS states and is discharged by `decide +kernel`. An internal node unfolds one row, simplifies all concrete guards, and rewrites by every imported child theorem. Table 1 records the fixed split.

No.	Four-term tail in (6.1)	L	Leaves	Result
1	(54, 56, 60, 4320)	45,360	30	false
2	(56, 60, 64, 320)	30,240	30	false
3	(56, 60, 80, 160)	15,120	30	false
4	(56, 60, 96, 120)	15,120	42	false
5	(56, 72, 90, 96)	15,120	42	false

Table 1: Split kernel DFS certificates for the five profiles.

There are 251 generated fiber modules in total: one fixed profile-data module, 174 leaves, 71 internal nodes, and five profile endpoints. The deterministic generator reconstructs the five trees and reproduces every module byte for byte. This generator chooses the split, but is not a proof premise: each leaf, each parent-to-child connection, and each profile endpoint is elaborated by Lean.

The supplementary primary Python program is deliberately separate from the Lean proof route. It reorders rows, quotients by box permutations, stores seen indices as sets, and applies a safe remaining-mass rejection. The separately implemented verifier keeps boxes labelled and uses bitsets and a different traversal. Neither implementation is imported by the final Lean theorem.

The optimized primary program and the separately implemented verifier agree on the following decision boundary. This implementation diversity is a reproducibility check, not an independent mathematical proof of the model.

Length	Arithmetic profiles	Fiber survivors	Survivor SHA-256 prefix
17	5	0	4f53cda18c2baa0c
18	470	39	589513a618402b33

For the empty list at seventeen, the displayed hash is the SHA-256 of the canonical JSON encoding []. The complete hashes and all thirty-nine length-eighteen lists are stored in the JSON artifact.

Proposition 6.1 is formalized as `kernelFiberDFS4_complete_of_orderedAssignment`, after `IndexFourFiberAssignment.toOrderedMaskAssignment4`. Thus the negative DFS certificates cannot miss an assignment forced by a partition.

Proposition 6.2 (Fiber rejection). *None of the five profiles in (6.1) admits an index-four fiber assignment.*

Proof. The Lean completeness theorem maps every such assignment to a branch on which `kernelFiberDFS4` returns true. The five split profile certificates prove that the same root searches return false. Their combination is `seventeenIndexFourAssignmentObstruction_kernel`; hence no assignment exists. $\square$

Proof of Theorem 1.1. Assume a counterexample with at most seventeen cells exists and choose one with the fewest cells. Lemma 2.2 and minimality first exclude index two. Apply Lemma 2.1 and replace the partition by its finite quotient with the same cell count and complete index profile. Sort the still-distinct indices. Equations (3.2) and (3.3), together with Lemma 4.1 and the four published propositions, make this list an arithmetic candidate.

If its length is at most sixteen, Proposition 5.1 and the pure-kernel empty-search computation give a contradiction. The remaining case has length seventeen. Exact evaluation places its profile among the five in (6.1). The index-four cell then constructs the three-box assignment above, contradicting Proposition 6.2. $\square$

The 39 length-eighteen survivors satisfy only the recorded arithmetic and first-order fiber conditions. They neither realize coset partitions nor show that an eighteen-cell counterexample exists.

7 Lean formalization and trust boundary

The repository uses Lean 4.30.0 and mathlib 4.30.0. The main local endpoints are:

checked_seventeen_survivors

the exact five-profile arithmetic boundary from the split D0–D17 certificates;

localMargolisSchnabelFacts

the locally proved six-field obstruction bundle;

kernelFiberDFS4_complete_of_orderedAssignment

completeness of the direct labelled depth-first search;

seventeenIndexFourAssignmentObstruction_kernel

the combined rejection of all five profiles;

erdos274AtMostSeventeen

the closed at-most-seventeen theorem with no external theorem parameter.

The exact source interface `MargolisSchnabelFacts` has six fields: Lemma 2.3(b,c) and detector-level corollaries of Propositions 4.2, 4.3, 4.5, and 4.7. It remains useful as an internal theorem map, but `localMargolisSchnabelFacts` proves every field locally. The final theorem takes no argument of this type.

The dependency chain also contains the normal-core finite quotient, exact cell preimages, partition descent, ambient and relative index preservation, harmonic subfamily selection, and detector-to-cell transfer. Thus no cited mathematical theorem is left as an external parameter of the bounded endpoint. This is not a claim that the published article has been transcribed line by line; the local modules prove the required source-shaped statements directly.

Both theorem-facing finite computations use split `decide +kernel` certificates. The arithmetic family has 18 depth modules and the fiber family has 251 generated modules. The parent theorems prove coverage by unfolding one transition and rewriting every successful child; a generator is not imported as an oracle.

For every displayed endpoint, including `erdos274AtMostSeventeen`, `#print axioms` reports only `propext`, `Classical.choice`, and `Quot.sound`. The release checker verifies this allowlist, the final closed signature with no external theorem parameter, and the absence of `sorry`, custom axioms, `native_decide`, `bv_decide`, native compiler equality, `unsafe`, `extern`, and `implemented_by` in the first-party theorem tree.

8 Literature-search scope and priority boundary

Margolis–Schnabel use the four harmonic obstructions in a computation organized by group order and prove the conjecture for groups of order below 1440 [8]. They do not state the complete cell-count enumeration or index-four fiber obstruction through seventeen.

Akman–Sissokho prove the arbitrary-group result through seven cells and report that their computation did not finish at eight [1]. Their later Steiner paper treats structural hypotheses rather than a larger unrestricted cell-count bound [2, 3]. Recent work on simple and symmetric groups also imposes group-class assumptions [5].

A bounded public-source search found no earlier arbitrary-group theorem for eight through seventeen cells. The search covered the maintained problem page, the primary papers above, public bibliographic indexes, and public citation graphs, but it was not exhaustive. In particular, no claim is made about all MathSciNet records, theses, non-English sources, private notes, or folklore. The defensible wording is therefore:

We present an unreviewed computer-assisted proof candidate that every nontrivial coset partition of an arbitrary group into at most seventeen cosets has multiplicity. To our knowledge, the previously published unrestricted bound was seven, due to Akman and Sissokho.

This qualifies the literature comparison, not the mathematical statement of Theorem 1.1. The bounded search does not establish priority.

9 Reproducibility and disclosure

The standalone repository contains the complete Lean import closure, the two deterministic kernel-certificate generators, and two pairs of separately implemented exact-arithmetic Python diagnostics. The paired diagnostic programs share the same mathematical specification but import no code from one another. The repository also contains two frozen JSON artifacts, short source records, this manuscript, and SHA-256 manifests. The command

`bash scripts/check_release.sh`

runs the artifact replays, byte-for-byte certificate checks, a two-job controlled Lean build, theorem and axiom audit, source scan, and manifest checks. A release decision should also include a build from a fresh copy without local Lean artifacts.

LLMs accessed through ChatGPT Pro and OpenAI Codex, primarily GPT-5.6, were used in proof exploration, formalization, test generation, literature screening, and editing. Rio Itabe is responsible for the mathematical claim, source use, code, manuscript, and release decisions. LLM output was not treated as a proof, bibliographic authority, or independent review.

References

- [1] F. Akman and P. A. Sissokho, *Transversal coset partitions of groups*, Beitr. Algebra Geom. **66** (2025), 417–441. doi:10.1007/s13366-024-00748-9.
- [2] F. Akman and P. A. Sissokho, *Steiner coset partitions of groups*, Canad. Math. Bull. **68** (2025), no. 4, 1359–1373. doi:10.4153/S0008439525100787.
- [3] F. Akman and P. A. Sissokho, *Steiner coset partitions of groups – erratum*, Canad. Math. Bull. **69** (2026), no. 2, 702. doi:10.4153/S0008439525101392.
- [4] T. F. Bloom, *Erdős Problem 274*, maintained problem record, accessed 16 August 2026. erdosproblems.com/274.
- [5] M. Garonzi and L. Margolis, *The Herzog–Schönheim conjecture for simple and symmetric groups*, arXiv:2509.25118v2 (2025; revised 2026), forthcoming in Algebra & Number Theory. arXiv:2509.25118.
- [6] M. Herzog and J. Schönheim, *Research problem no. 9*, Canad. Math. Bull. **17** (1974), 150.
- [7] I. Korec and Š. Známl, *On disjoint covering of groups by their cosets*, Math. Slovaca **27** (1977), no. 1, 3–7. dml.cz/dmlcz/129020.
- [8] L. Margolis and O. Schnabel, *The Herzog–Schönheim conjecture for small groups and harmonic subgroups*, Beitr. Algebra Geom. **60** (2019), 399–418. doi:10.1007/s13366-018-0419-1.